Introduction to Databases (IDB)

Assignment Submission

Name	Roll No	Cohort
Rani Kumari	B25BS1270	Cohort 3

Task 1

Users Table

Field	Description / Example
User ID	Unique number identifying each user (e.g., 1, 2, 3)
Name	Full name of the user (e.g., Rani Kumari)
Username	Unique handle used to log in / be searched (e.g., rani_k)
Email	Contact email address of the user

User ID	Name
1	Aman
2	Riya
3	Neha

Posts Table

Field	Description / Example
Post ID	Unique number identifying each post
User ID	ID of the user who created the post (links to Users table)
Caption	Text written by the user describing the post
Created Date/Time	Date and time when the post was uploaded

Post ID	User ID	Caption
101	1	First Post
102	1	Travel Day
103	2	Good Morning

Likes Table

Field	Description / Example
Like ID	Unique number identifying each like
User ID	ID of the user who liked the post
Post ID	ID of the post that was liked
Liked Date/Time	Date and time when the like was given

Like ID	User ID	Post ID
1	2	101
2	3	101
3	1	103

Task 2

Design of the Users table for the application:

Column	Description
User ID (PK)	Uniquely identifies each user
Name	Full name of the user
Username	Unique login handle
Email	Contact email address

a. Which column should uniquely identify each user? The User ID column should uniquely identify each user.

b. Why should this value be unique? It should be unique so that every user has a distinct identity in the database. This avoids confusion between two different users and makes sure that when other tables (like Posts or Likes) refer to a User ID, they always point to exactly one, correct user.

c. Can two users have the same User ID? Explain briefly. No. Since User ID is the primary key of the Users table, it must be unique for every row. If two users had the same User ID, the database would not be able to tell them apart, which could lead to wrong data being linked to the wrong person.

d. Mention one additional column that could be useful in the Users table. A 'Date of Joining' column (or a 'Profile Picture' / 'Password' column) could be useful to record when the user created their account.

Task 3

Design of the Posts table, connected to the user who created it:

Column	Description
Post ID (PK)	Uniquely identifies each post
User ID (FK)	Identifies which user created the post
Caption	Text description of the post
Created Date/Time	When the post was created

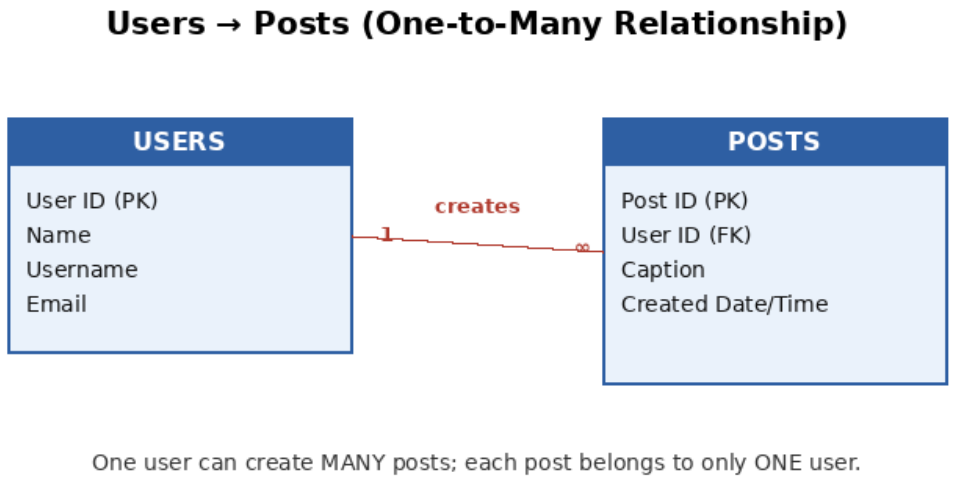
a. Which column uniquely identifies a post? The Post ID column uniquely identifies a post.

b. Which column connects the post to the Users table? The User ID column in the Posts table connects each post back to the user who created it in the Users table.

c. Can one user create multiple posts? Yes, one user can create multiple posts. Each post will simply have the same User ID for that user, but a different, unique Post ID.

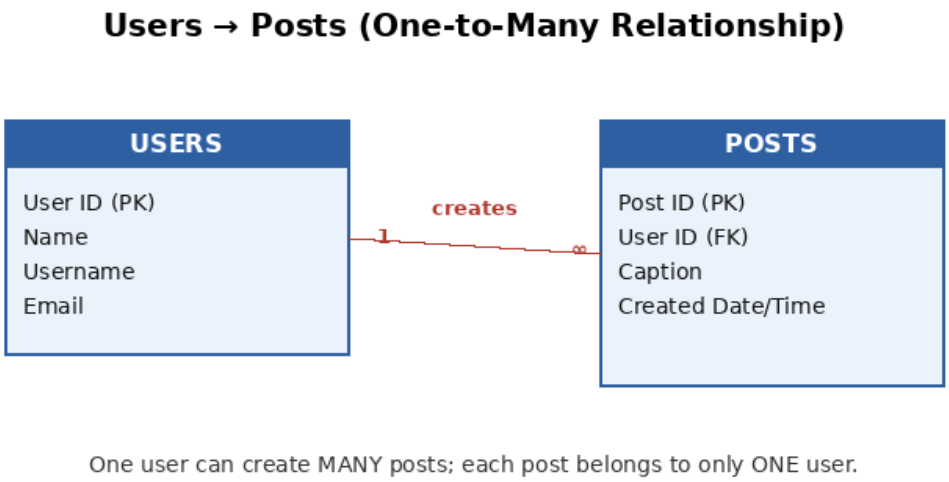
d. What type of relationship exists between Users and Posts? A One-to-Many relationship exists: one user can have many posts, but each post belongs to only one user.

Relationship between Users and Posts



Task 4

Relationship created: Users → Posts (see diagram above, also reproduced below for this task).



a. Identify the One side of the relationship. The Users table is on the One side of the relationship.

b. Identify the Many side. The Posts table is on the Many side of the relationship.

c. Explain the meaning of a One-to-Many relationship in your own words. A One-to-Many relationship means that a single record in one table (here, one user) can be linked to several records in another table (many posts), but each record in the second table is linked back to only one record in the first table. So one user can own many posts, but a post cannot belong to more than one user.

d. Give one more real-life example of a One-to-Many relationship. One teacher can teach many students in a class, but each student in that class has only one class teacher. Another example is one customer can place many orders, but each order belongs to only one customer.

Task 5

Now consider the Likes feature. A user can like many posts, and one post can be liked by many users.

Column	Description
Like ID (PK)	Uniquely identifies each like
User ID (FK)	Identifies which user gave the like
Post ID (FK)	Identifies which post received the like

a. Why does the Likes table need both User ID and Post ID? It needs both because a single 'like' is only meaningful as a connection between a specific user and a specific post. User ID alone would not tell us which post was liked, and Post ID alone would not tell us who liked it. Together they record exactly who liked what.

b. What does one row in the Likes table represent? One row represents a single like event — one particular user liking one particular post.

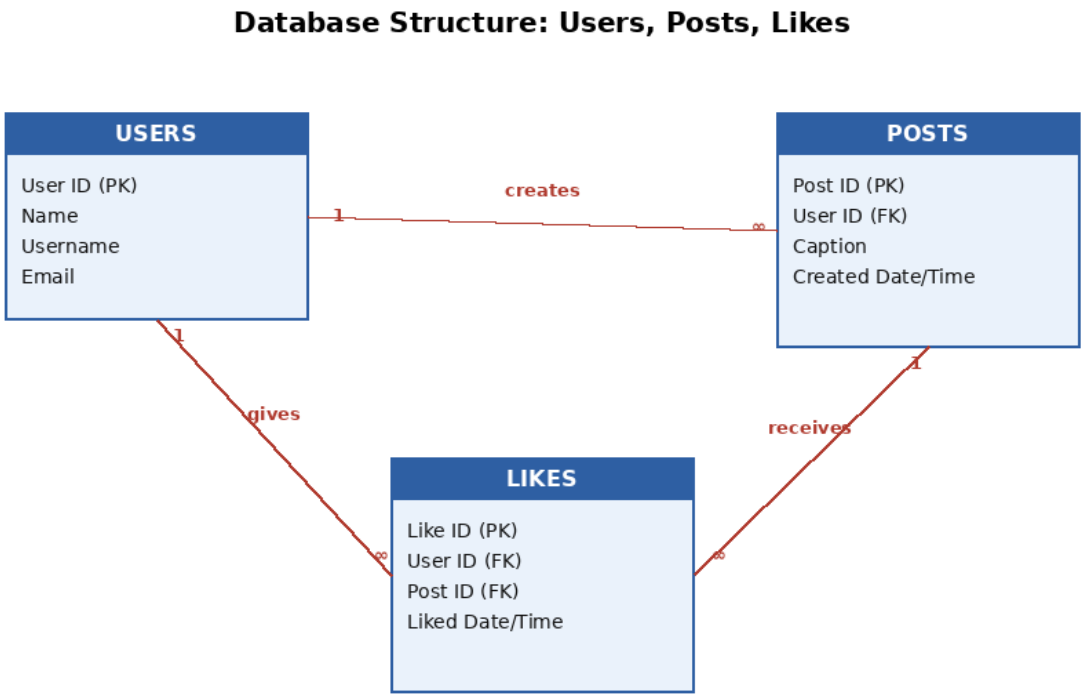
c. Can one user like multiple posts? Yes. The same User ID can appear in multiple rows of the Likes table, each paired with a different Post ID.

d. Can one post receive likes from multiple users? Yes. The same Post ID can appear in multiple rows of the Likes table, each paired with a different User ID.

e. What type of relationship exists between Users and Posts when likes are considered? A Many-to-Many relationship exists: one user can like many posts, and one post can be liked by many users. The Likes table is the junction table that makes this possible.

Task 6

Complete database structure showing Users, Posts, and Likes, with the relationships between all three tables:



The diagram above clearly shows:

- Users to Posts — One-to-Many (a user creates many posts)
- Users to Likes — One-to-Many (a user gives many likes)
- Posts to Likes — One-to-Many (a post receives many likes)

a. Which tables have a One-to-Many relationship? Users–Posts, Users–Likes, and Posts–Likes are all One-to-Many relationships.

b. How does the Likes table help represent a Many-to-Many relationship between Users and Posts? The Likes table sits between Users and Posts as a junction (bridge) table. Instead of connecting Users directly to Posts in a many-to-many way, it splits the relationship into two simple one-to-many links: one user can have many rows in Likes, and one post can have many rows in Likes. Together, this lets any user be linked to any number of posts, and vice-versa.

c. Why is a separate Likes table useful instead of storing all likes directly inside the Posts table? If we tried to store every like inside the Posts table itself, we would need a separate column for every user who liked that post — but the number of likes is unknown and can keep growing. This would create repeating groups and wasted/empty columns, and updating them would be messy. A separate Likes table instead lets us add a new row for every new like, however many there are, without ever changing the structure of the Posts table.

Note: The relationships above were designed based on the described structure (equivalent to a dbdiagram.io export); the diagram is included as the required screenshot of the design.

Task 7

Given sample data:

Users

User ID	Name
1	Aman
2	Riya

User ID	Name
3	Neha

Posts

Post ID	User ID	Caption
101	1	First Post
102	1	Travel Day
103	2	Good Morning

Likes

Like ID	User ID	Post ID
1	2	101
2	3	101
3	1	103

a. Which user created Post 101? Post 101 has User ID 1, which belongs to Aman. So Aman created Post 101.

b. How many posts were created by Aman? Aman (User ID 1) created 2 posts — Post 101 ("First Post") and Post 102 ("Travel Day").

c. Which users liked Post 101? Looking at the Likes table, Post 101 was liked by User ID 2 (Riya) and User ID 3 (Neha).

d. Which post was liked by Aman? Aman has User ID 1. In the Likes table, User ID 1 liked Post ID 103 ("Good Morning").

e. Which relationship helps you connect Posts with Users? The User ID column (foreign key) inside the Posts table, which links each post back to the Users table — this is the One-to-Many relationship between Users and Posts.

f. Which table helps you identify who liked which post? The Likes table helps identify who liked which post, since each row pairs a User ID with a Post ID.

Task 8

A new application allows users to create an account, create multiple posts, like multiple posts, and follow multiple users. We already have the Users, Posts, and Likes tables. Now consider the Follow feature.

a. Would the Follow feature require a separate table? Why? Yes. Following is a many-to-many relationship among users themselves — one user can follow many other users, and one user can be followed by many other users. This cannot be captured with a single column in the Users table, so a separate junction table (similar to the Likes table) is needed to store each follow connection as its own row.

b. Suggest suitable columns for a Follow table. Follow ID (primary key), Follower User ID (the user who follows), Following User ID (the user being followed), and Followed Date/Time (when the follow happened).

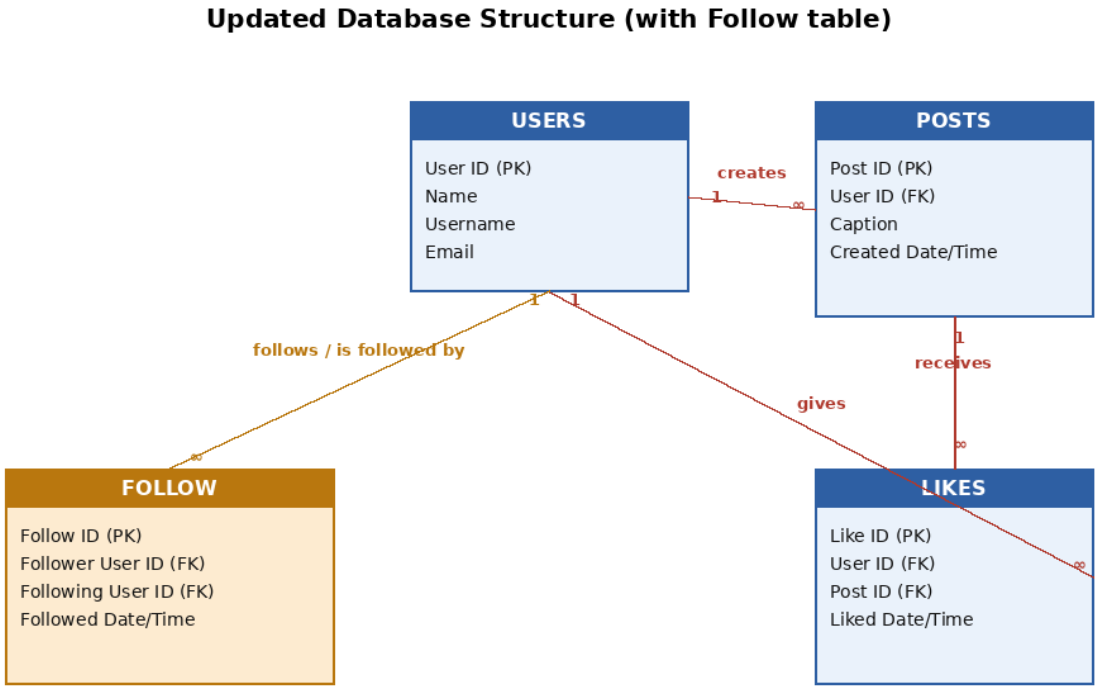
c. If User A follows User B, what information should be stored? A new row should be stored with Follower User ID = A and Following User ID = B (along with a new unique Follow ID and, optionally, the date/time of the follow).

d. Can one user follow many users? Yes, one user can follow many other users — their User ID will appear multiple times in the Follower User ID column, each time paired with a different user being followed.

e. Can one user be followed by many users? Yes, one user can be followed by many other users — their User ID will appear multiple times in the Following User ID column, each time paired with a different follower.

f. What type of relationship does this represent? This represents a Many-to-Many relationship, and since it connects the Users table to itself, it is also called a self-referencing relationship.

g. Updated database diagram including the Follow table



Follow is self-referencing on Users: one user can follow many, and be followed by many.

Summary

The Users, Posts, Likes, and Follow tables work together to store all the information this social media application needs. The Users table holds each person's basic details and is the anchor that every other table connects back to. Each row in the Posts table stores one post along with the User ID of the person who created it, giving a One-to-Many relationship between Users and Posts. The Likes table connects Users and Posts in a Many-to-Many way by storing one row for every single like, pairing a User ID with a Post ID. Similarly, the Follow table connects the Users table to itself, storing one row for every follow relationship as a Follower User ID and a Following User ID. Together, these four tables avoid duplicating data and instead use IDs to link related information, which keeps the database organised, scalable, and easy to query as the number of users, posts, likes, and follows keeps growing.